

# New Problems, New Paradigms

Mark Priestley

# Programming Styles, Language Paradigms

What are they?

Where do they come from?

This talk is a historical case study –  
looking for the origins of functional  
programming

# Practitioners' history

## Conception, Evolution, and Application of Functional Programming Languages

PAUL HUDAK

*Yale University, Department of Computer Science, New Haven, Connecticut 06520*

Indeed the lambda calculus is usually regarded as the first functional language, although it was certainly not thought of as programming language at the time, given that there were no computers on which to run the programs. In any case, modern functional languages can be thought of as (nontrivial) embellishments of the lambda calculus.

## Some History of Functional Programming Languages

D. A. Turner

The first functional programming language and the second oldest programming language still in use (after FORTRAN), LISP began life in 1958 as a project led by John McCarthy at MIT. The aim was to create a system for programming computations over symbolic data, starting with an algorithm McCarthy had drafted for symbolic differentiation. The first published account of the language

# Theory → Practice

A traditional view of technological innovation. But:

FP (now) theoretically rooted in  $\lambda$ -calculus

≠

FP historically derived from  $\lambda$ -calculus

Real history is messier (and more interesting)

THE CHESS MACHINE: AN EXAMPLE OF DEALING  
WITH A COMPLEX TASK BY ADAPTATION

Allen Newell

P-620

28 December 1954



The term "complex information processing" has been chosen to refer to those sorts of behaviors -- learning, problem solving, and pattern recognition -- which seem to be incapable of precise description in any simple terms, or perhaps, in any terms at all. The advent of the high-speed electronic digital

The problem of playing good chess certainly falls into this class of ultracomplex problems. It is a useful type case for general discussion because the nature of the task and the complexities surrounding it are common knowledge. Further, it already has something of a history (1,2,3,4).

# THE LOGIC THEORY MACHINE A COMPLEX INFORMATION PROCESSING SYSTEM

Allen Newell and Herbert A. Simon<sup>1</sup>  
The RAND Corporation, Santa Monica, Calif.  
and the Carnegie Institute of Technology,  
Pittsburgh, Pa.

## Abstract

In this paper we describe a complex information processing system, which we call the logic theory machine, that is capable of discovering proofs for theorems in symbolic logic. This system, in contrast to the systematic algorithms that are ordinarily employed in computation, relies heavily on heuristic methods similar to those that have been observed in human problem solving activity. The specification is written in a formal language, of the nature of a pseudo-code, that is suitable for coding for digital computers. However, the present paper is concerned exclusively with specification of the system, and not with its realization in a computer.



at least be a little more systematic about what constitutes a complex information process.

1. A complex process consists of very large numbers of subprocesses, which are extremely diverse in their nature and operation. No one of them is central or, usually, even necessary.
2. The elementary component processes need not be complex; they may be simple and easily understood. The complexity arises wholly from the pattern in which these processes operate.
3. The component processes are applied in a highly conditional fashion. In fact, large numbers of the processes have the function of determining the conditions under which other processes will operate.



# Programming the Logic Theory Machine\*

A. NEWELL<sup>†</sup> AND J. C. SHAW<sup>†</sup>

## *Flexibility of Memory Assignment:*

1) There should be no restriction on the number of different lists of items of information to be stored. This number should not have to be decided in advance; that is, it should be possible to create new lists at will during the course of computation.

2) There should be no restriction on the nature of the items in a list. These might range from a single symbol or number to an arbitrary list. Thus, it should be possible to make lists, lists of lists, lists of lists of lists, etc.

## *Flexibility in the Specification of Processes:*

1) It should be possible to give a name to any sub-routine, and to use this name in building other sub-routines. That is to say, there should be no limitation on the size and complexity of hierarchies of definitions.

3) It should be possible to define processes implicitly; *e.g.*, by recursion. More generally, the programmer should be able to specify any process in whatever way occurs naturally to him in the context of the problem.

# Lists

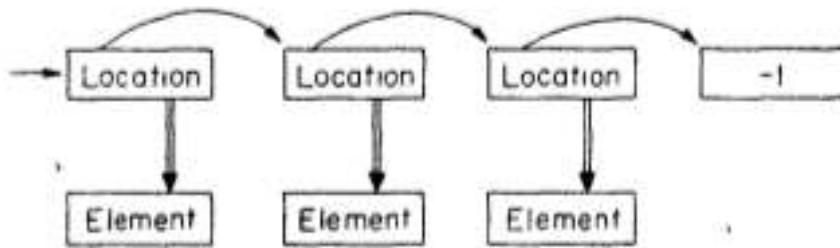


Fig. 4—List of elements. The left half of the location word contains the address of the next location word (single arrow); the right half contains the address of the element (double arrow).

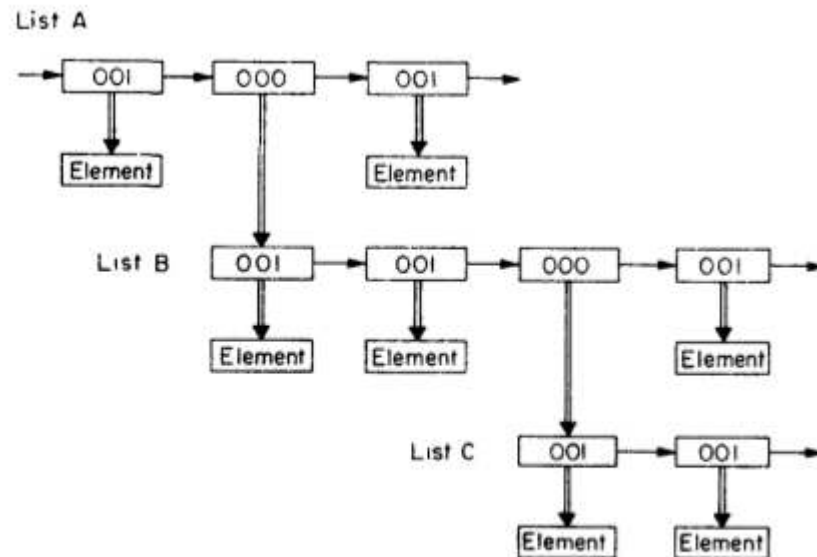
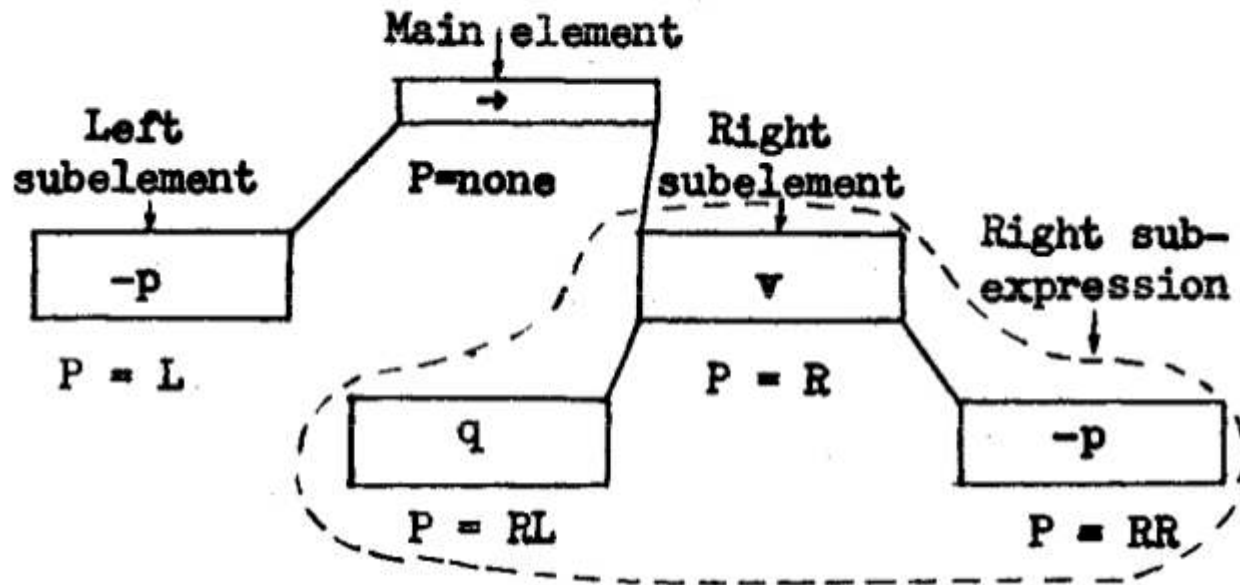


Fig. 5—General list. List B is the second item on list A; list C is the third item on list B.

# Representing expressions

1.7     $\neg p \rightarrow q \vee \neg p$  ((not-p) implies (q or not-p))

The entire sequence is the expression, X(1.7). It consists of the elements  $\neg p$ ,  $\rightarrow$ ,  $q$ ,  $\vee$ ,  $\neg p$ . The expression may be written in "tree" form, as follows, where the rectangles indicate the elements:



# Recursion

A OPER L C R B Seg.

Compare expressions

Compare  $X(x)$  with  $X(y)$ ; if they match,  $\rightarrow b$ .

CX x y b

|     |     |   |    |
|-----|-----|---|----|
| CGG | L C | B | T  |
| CGG | C L | B |    |
| TV  | L   | A |    |
| TV  | C   | B |    |
| -CC | L C | B |    |
| FL  | L 1 |   | CX |
| FL  | C 2 |   |    |
| -CX | 1 2 | B |    |
| FR  | L 3 |   |    |
| FR  | C 4 |   |    |
| -CX | 3 4 | B |    |
| BHB |     |   |    |

$G(L) = G(R)$ , otherwise  $\rightarrow B$ .

$C(L) = C(R)$

Recursion down tree of expressions.

|   |     |     |   |    |
|---|-----|-----|---|----|
| A | -TV | C   | B | CN |
|   | -CN | L C | B |    |
|   | BHB |     |   |    |

L and C both variables;  
with identical names.

B BUN

A PROPOSAL FOR THE  
DARTMOUTH SUMMER RESEARCH PROJECT  
ON ARTIFICIAL INTELLIGENCE

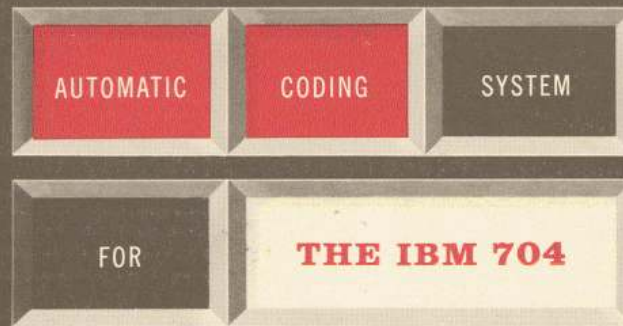
J. McCarthy, Dartmouth College  
M. L. Minsky, Harvard University  
N. Rochester, I. B. M. Corporation  
C. E. Shannon, Bell Telephone Laboratories

We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be

# Programming

- Newell and Simon are the only people at Dartmouth to have a working program
- Inspires others to start similar projects
- Nobody likes IPL
- but there's another new language in town in 1956

# Fortran



An “algebraic language”

## A FORTRAN-COMPILED LIST-PROCESSING LANGUAGE

H. Gelernter, J. R. Hansen, and C. L. Gerberich  
IBM Research Center, Yorktown Heights, New York

When the present authors embarked upon their effort to simulate a geometry theorem-proving machine, it was early decided that an NSS organization of memory would best serve their purpose, and consideration was given to the translation of a JOHNNIAC IPL for use with the IBM 704 computer. However, J. McCarthy, who was then consulting for the project, suggested that Fortran could be adapted to serve the same purpose. He pointed out that the nesting of functions that is allowed within the Fortran format makes possible the construction of elaborate information-processing subroutines with a single statement. The authors have since discovered a further substantial advantage of an algebraic list-processing language such as one written within the framework of Fortran. It is the close analogy that exists between the structure of an NSS list and a certain class of algebraic expressions that may be written within the language. Not to be overlooked is the considerable sophistication built into the Fortran compiler itself, all



### III. *The FORTRAN List-Processing Functions*

FORTRAN is in itself an information processing language of great versatility and sophistication. Our list-processing functions merely serve to increase the “vocabulary” of the language so that list manipulation processes may be described within the FORTRAN framework as are ordinary computer processes. We are thus able to enjoy the same ease of programming, ease of modification, and extensive debugging aids available to the programmers of standard numerical problems.

The following FLPL statement will interchange the  $y$ -components of points A and B in LPTS:

$$\text{JUNK} = \text{XSTORAF}(\underbrace{\text{XCDADF(LPTS)}}_{\beta 2}, \underbrace{\text{XSTORAF}(\underbrace{\text{XCDAF(LPTS)}}_{\alpha 2}, \underbrace{\text{XCADADF(LPTS)}}_{\beta 21})}_{\alpha 21})$$

The value of the entire function is  $\beta 21$ , the address of the  $y$ -component of B. Since we do not wish to further process this number, it is discarded into a “bottomless pit” by setting the function equal to a standard variable, “JUNK”.

But no recursion.

# Two familiar functions

$\text{XCDRF}(J)$ ,  $\text{XCARF}(J)$ ,  $\text{XCPRF}(J)$ ,  $\text{XCSPF}(J)$ ,  $\text{XCTRF}(J)$ . Extract contents of the (decrement, address, prefix, signed prefix, tag) register of the word stored in location  $J$ .

- (1)  $\text{XCARF}(\text{LPTS}) \Rightarrow \alpha 1$ , the name of the first point on LPTS (note that the compiler substitutes the machine name  $\alpha$  for LPTS).
- (2)  $\text{XCARF}(\text{XCDRF}(\text{LPTS})) \Rightarrow \beta 1$ , the name of the second point on LPTS.

$\text{XCARNF}(J)$ ,  $\text{XCDRNF}(J)$ , where  $n$  is an integer  $\leq 9$ . Equivalent to  $n$  iterations of the base function; thus  $\text{XCAR3F}(J)$  is equivalent to  $\text{XCARF}(\text{XCARF}(\text{XCARF}(J)))$ .

$\text{XCADF}(J)$ ,  $\text{XCADAF}(J)$ ,  $\text{XCADADF}(J)$ ,  $\text{XCDAF}(J)$ ,  $\text{XCDADF}(J)$ ,  $\text{XCDADAF}(J)$ . Equivalent to a sequence of alternations of  $\text{XCARF}$  AND  $\text{XCDRF}$ , specified by the sequence of “D’s” and “A’s” in the name of the function; thus  $\text{XCDADF}(J)$  is equivalent to  $\text{XCDRF}(\text{XCARF}(\text{XCDRF}(J)))$ .

# McCarthy and chess

- 1957-8: chess moves
  - Written in Fortran
  - Limitations of Fortran's IF statement
- “Conditional expression”
  - A Fortran subroutine XIF(M, N1, N2)
  - Value N1 if  $M = 0$ , N2 otherwise
  - Always evaluates N1 and N2

# Symbolic differentiation

- Summer 1958
  - Uses lists to represent algebraic expressions
  - Used conditional expressions to define recursive functions
- Needs to apply a function to every element
  - *Maplist*: needs to take a function as an argument

# New techniques

- Innovations between 1956 and 1958:
  - Lists
  - Fortran's algebraic notation
  - Recursion
  - Conditional expressions
  - Lambda expressions
- Motivated by *different* practical programming problems
- Then put together in a single new language

# Lisp Memo 1 : September 1958

## AN ALGEBRAIC LANGUAGE FOR THE MANIPULATION OF SYMBOLIC EXPRESSIONS

by John McCarthy

### 1.2. Features of the language

1.2.1. Expressions are represented by lists

1.2.2. The convenience of algebraic notation

1.2.3. It is frequently convenient to define certain processes by recursion. This means that the program defining a subroutine may use the subroutine itself. This presents

1.2.4. Conditional expressions.

1.2.5. The use of functions and predicates as parameters

# LISP: A PROGRAMMING SYSTEM FOR SYMBOLIC MANIPULATIONS

by John McCarthy, MIT Computation Center

1959

Definition: An S-expression is either an atomic symbol or an expression of the form  $(e_1.e_2)$  where  $e_1$  and  $e_2$  are S-expressions. Examples: ABSA and  $(A.B)$  and  $(A.((C.D).A))$ .

A list  $(i_1, i_2, \dots, i_n)$  can be represented by the S-expression  $(i_1.(i_2.(\dots(i_n.NIL)\dots)))$ , that is  $(A,B,C)$  is represented as  $(A.(B.(C.NIL)))$ . Actually, our read routines will read the list

The three functions car, cdr, and cons are defined by the relations  $\text{car}[(e_1.e_2)] = e_1$ ,  $\text{cdr}[(e_1.e_2)] = e_2$ , and  $\text{cons}[e_1;e_2] = (e_1.e_2)$ . car and cdr are not defined for atomic arguments. As examples we have  $\text{car}[(A.B).C] = (A.B)$  and  $\text{cdr}[(A.B).C] = C$  and  $\text{cons}[(A.B);C] = ((A.B).C)$ .

S-expressions are represented in the IBM 704 computer by list structures similar to those used in the IPL's of Newell, Shaw and Simon.\* An S-expression is represented by

# Memo 4 : Oct 1958

## 2. A Revised Version of "Maplist"

### 2.2 Functional abstraction - The Church $\lambda$ 's.

In order to be able to use forms in the definition of the functions which are the arguments of maplist it is necessary to use functional abstraction as developed by Church. We digress from the subject of programming into mathematical logic in order to explain the idea of functional abstraction.



# Symbolic differentiation

The routine for differentiating the expression represented by the list structure starting in location  $L$  with respect to the variable  $V$  is:

```
diff(L,V) = (L = V → C1, car(L) = 0 → C0, car(L) = plus →  
cons(plus, maplist(cdr(L), λ(J, diff(car(J), V)))), car(L) = times →  
cons(plus, maplist(cdr(L), λ(J, cons(times, maplist(cdr(L), λ(K,  
(J ≠ K → copy(car(K)), 1 → diff(car(K), V))))))), 1 → error)
```

# Lisp not purely functional

PROGRAMS IN LISP  
by John McCarthy

LISP. Its objective is to add to the system of that report a program feature. This takes the form of allowing functions to be defined by programs including sequences of Fortran-like statements, eg.

```
y=cons[ff[subst[A;y;z]];(A,B)]
```

Since all computable functions can be expressed in LISP without the program feature, this feature can only be regarded as a convenience. However, it is a convenience at which we cannot afford to sneer.

(By 1964, Lisp programmers distinguished two programming styles, depending on whether the “prog” feature was used or not.)

# Lisp's Theoretical Foundations

March 4, 1959

Artificial Intelligence Project--RLE and MIT Computation Center

Memo 8

RECURSIVE FUNCTIONS OF SYMBOLIC EXPRESSIONS AND THEIR COMPUTATION

BY MACHINE

by J. McCarthy

The attached paper is a description of the LISP system starting with the machine-independent system of recursive functions of symbolic expressions. This seems to be a better point of view for looking at the system than the original programming approach. After revision, the paper will be submitted for publication in a logic or computing journal.

This memorandum contains only the machine independent parts of the system. The representation of S-expressions in the computer and the system for representing S-functions by computer subroutines will be added.

# Lambda Calculus

- Lisp's theoretical foundations are *not* the lambda calculus
- The idea of the lambda calculus proved very fertile, however
  - Approach to programming language semantics
  - Languages based on it (Peter Landin)
  - Now assumed to be the foundation of the functional paradigm

# Where do paradigms come from?

From practice, not from theory:

1. A specific **problem** (how to program complex information processes)
2. Programmers come up with new techniques to **solve** the problems (lists, recursion)
3. And package these up in new **languages** that makes them convenient to use (IPL, Lisp)
4. (Come up with a **computational model** which “crystallizes” the new ideas (lambda calculus))

# Example : object-orientated paradigm

- Problem: discrete event simulation
- Technique: objects, messages
- Languages
  - Simula (extended Algol 60)
  - Smalltalk
- Computational model?